

Military Institute of Science and Technology

Department of Computer Science and Engineering

CSE 308 – Operating System Sessional

Predictive Exam-Aware Resource Manager

Technical Project Report

Group Members:

Najifa Raksanda (Roll: 202414010)

Aria Nawshin Ashka (Roll: 202414021)

Swagata Mallick (Roll: 202414031)

Chowdhury Shah Noor Binte Jamal (Roll: 202414039)

September 2026

Abstract

Computer-based examinations need a way to distinguish permitted workloads from prohibited applications while keeping essential tasks responsive. Predictive Exam-Aware Resource Manager is a Linux user-space service that samples processes and memory, applies an exam policy, classifies memory pressure, and records decisions for a teacher-facing local dashboard. It uses process identity rather than a name alone when considering enforcement, reads memory Pressure Stall Information (PSI), and uses a rule-based predictor with hysteresis. Its default `detect_only` mode records proposed actions without terminating processes or writing resource limits. Optional signal and cgroup v2 enforcement is limited to controlled tests. The report explains the operating-system mechanisms, implementation, evaluation procedure, evidence requirements, and limitations. It separates implemented behavior from proposed future improvements and does not claim unverified performance results.

Contents

Abstract	i
1 Introduction	1
1.1 Problem and motivation	1
1.2 Objectives	1
1.3 Scope and course alignment	1
2 Background and Relevant OS Concepts	1
2.1 Process management	1
2.2 CPU scheduling and process priority	2
2.3 Memory management	2
2.4 Pressure Stall Information	2
2.5 The Linux <code>/proc</code> filesystem	2
2.6 Linux signals	3
2.7 cgroup v2 and resource allocation	3
2.8 Background monitoring and synchronization	3
2.9 SQLite and persistent event logging	3
2.10 Prediction and resource-pressure detection	4
2.11 OS concept summary	4
3 System Design and Methodology	5
3.1 Architecture	5
3.2 Policy and decision method	5
3.3 Sampling and baseline	5
3.4 Safety boundary	5
4 Implementation	6
4.1 Modules and responsibilities	6
4.2 Data and interfaces	6

5	Experiments and Evaluation	6
5.1	Environment and reproducible procedure	6
5.2	Experiment matrix	7
5.3	Metrics and interpretation	7
5.4	Results and evidence status	8
6	Challenges and Discussion	8
6.1	Individual contributions	9
7	Limitations	9
8	Conclusion	10
9	Future Work	10
A	Reproduction Checklist	10

1 Introduction

1.1 Problem and motivation

During a programming examination, a student may legitimately run a compiler, editor, shell, and language runtime. The same computer may also run prohibited applications or resource-intensive workloads. A simple blocklist does not explain why an action was selected, and immediate termination risks disrupting legitimate work. Ordinary Linux process and resource controls do not provide an exam-specific policy view for a teacher. This project adds a local, explainable policy and evidence layer above the operating system. It is not an operating system, kernel extension, or replacement scheduler.

1.2 Objectives

The implemented objectives are to (1) observe process identity and resource use, (2) classify processes as allowed, blocked, protected, or unknown under a validated JSON policy, (3) measure memory usage, growth, PSI, and optional cgroup metrics, (4) classify pressure and explain the resulting policy decision, (5) record events and samples persistently, (6) provide a live local dashboard and exports, and (7) keep enforcement optional and guarded. The project prioritizes safe demonstration over automatic intervention.

1.3 Scope and course alignment

The strongest direct course links are processes, threads and synchronization, and memory management. Linux signals and cgroup v2 provide concrete process-control and resource-control mechanisms. HTTP communication and filesystem interfaces provide supporting examples of inter-process communication (IPC) and file systems. The project does not implement a CPU scheduling algorithm, a deadlock algorithm, a disk scheduling algorithm, or a hypervisor. VirtualBox is used only as an isolated test environment. This distinction follows the course outline's Part A and Part B topic list [1].

2 Background and Relevant OS Concepts

2.1 Process management

A process is an executing program with its own virtual address space and kernel-maintained state. Linux assigns each process a process identifier (PID), exposes process-specific information under `/proc/<PID>/`, and tracks lifecycle states such as running, sleeping, and exited [3]. A process name is useful for policy display but is not a strong identity: applications can have similar names, a name can be changed, and a PID can be reused after a process exits. In this implementation, `ProcessMonitor` collects the PID, name, executable path when accessible, creation time, CPU percentage, resident memory (RSS), and status using `psutil`. It compares the pair (PID, creation time) between samples to recognize a new process. If a process disappears, it is removed from the current snapshot; before a destructive action, the action manager checks that the live PID still has the expected creation time. This reduces, but cannot eliminate, time-of-check/time-of-use races.

2.2 CPU scheduling and process priority

CPU scheduling selects which runnable task receives processor time. Linux offers scheduling classes and priority-related controls; for ordinary tasks, the `nice` value can influence relative CPU share under contention [4]. These are kernel scheduling mechanisms. This project records sampled process CPU use, but its *policy priority* is a separate label (for example, low, normal, or high) assigned by JSON configuration. The decision engine uses that label to choose protection or a resource-control recommendation during high memory pressure. It does not read or change Linux `nice` values, implement a scheduling policy, or directly allocate CPU time. Thus, policy priority contributes to resource-management decisions without being a CPU scheduler.

2.3 Memory management

The OS manages physical RAM, virtual address spaces, page mappings, and reclamation. An individual process's RSS approximates the physical pages currently resident for that process; it is not the full virtual address-space size. In system mode the project's `MemoryMonitor` obtains total and available memory through `psutil`, while Linux exposes related system data through `/proc/meminfo`. Growth is calculated from changes in measured used memory over elapsed time. With an explicitly configured cgroup source, the monitor instead reads files such as `memory.current`, `memory.max`, and `memory.events`. Exam workloads may experience latency or out-of-memory failure under heavy contention, so both quantity of memory used and signs of stalled work matter. High usage alone does not prove harmful pressure.

2.4 Pressure Stall Information

Linux PSI reports the fraction of time tasks were delayed by contention for CPU, memory, or I/O resources [6]. A `some` line means at least one task was stalled during the measurement interval; `full` means all non-idle tasks in the measured scope were stalled. The Linux files expose rolling averages, including `avg10`. This project reads *memory* PSI from `/proc/pressure/memory`, or from a configured cgroup's `memory.pressure`; it does not integrate CPU or I/O PSI into its predictor. Memory PSI complements RAM percentage: a machine can have substantial memory use with little stalling, or experience contention before a simple percentage threshold is reached. The predictor includes the larger of memory `some avg10` and `full avg10` as a pressure signal.

2.5 The Linux `/proc` filesystem

`/proc` is a virtual filesystem generated by the kernel, not an ordinary collection of persistent disk files [3]. Per-process directories such as `/proc/<PID>/` contain files including `status`, `stat`, and `cmdline`; system-level files include `/proc/meminfo` and `/proc/pressure/memory`. The process library obtains Linux process and system data through OS interfaces, while the project's PSI reader accesses the pressure file directly. The advantages are a standard, inspectable source and no custom kernel module. Limitations include permissions, short-lived processes disappearing between reads, PID reuse, and metrics whose meanings differ from simple application-level totals.

2.6 Linux signals

A signal is a kernel-mediated notification to a process. `SIGTERM` asks a process to terminate and permits its handler to clean up; `SIGKILL` cannot be caught or ignored [5]. In `detect_only` mode the project records a termination recommendation but sends no signal. In guarded enforcement mode it validates the target, sends `SIGTERM`, waits for a configured grace period, and may send `SIGKILL` if the process does not exit. The stronger fallback is therefore possible and should not be described as a `SIGTERM`-only implementation. PID 0, PID 1, the service itself, its parent, and stale process identities are rejected. Even a successful signal send is not equivalent to verified process exit.

2.7 cgroup v2 and resource allocation

Control groups provide kernel-enforced accounting and resource control for a group of processes. The unified cgroup v2 hierarchy exposes configuration and measurements as files [7]. Monitoring reads values such as `memory.current`, `memory.max`, `memory.events`, and `memory.pressure`; enforcement requires suitable permissions, a prepared managed hierarchy, placement of a target PID in `cgroup.procs`, and writes to controls. In this project a restricted managed group may receive `memory.high` and `memory.max`; a protected group may receive `memory.low`. These mechanisms are not equivalent to CPU scheduling. The default detect-only mode performs no privileged cgroup writes. Controller availability, delegation, kernel configuration, and permissions affect whether the optional enforcement path can run. A dashboard readiness indicator by itself does not prove that a process was moved or constrained.

2.8 Background monitoring and synchronization

The service starts a daemon monitoring thread when Exam Mode begins. Each loop samples processes and memory, predicts pressure, selects decisions, records events, and sleeps for the remaining configured interval. Flask handles local dashboard requests while the monitor continues. Both paths access current state, so unsynchronized updates could produce inconsistent reads. A Python `threading.RLock` protects shared snapshots and status; it is re-entrant so a thread that already holds it can safely enter code that acquires it again. Synchronization is a concurrency concept, not evidence that a deadlock prevention algorithm has been implemented.

2.9 SQLite and persistent event logging

The latest on-screen snapshot exists in memory and disappears when the service restarts. SQLite stores policy events and resource samples across requests and sessions. Event records include process identity, classification, recommended action, attempted/success flags, and timing; sample records include memory, PSI, pressure level, risk score, loop duration, and monitor overhead. Write-ahead logging supports dashboard reads while the monitor writes [8]. CSV and JSON exports allow later inspection. Logging supports auditability, but an unauthenticated local application and ordinary files do not provide strong tamper resistance.

2.10 Prediction and resource-pressure detection

The name “Predictive” refers to explainable rule-based early warning, not a trained machine-learning model. The predictor combines current memory percentage, positive growth rate, memory PSI, and cgroup out-of-memory events to form a raw level: Normal, Elevated, High, or Critical. Its configurable thresholds are in `config/policy.json`. The default High rules include memory at least 82%, growth at least 60 MiB/s, or PSI `avg10` at least 2%; Critical rules include 92%, 120 MiB/s, PSI at least 10%, or a new OOM-kill event. A stable level uses consecutive samples (two for escalation and three for recovery by default) to reduce oscillation. A 0–100 risk score summarizes signal strength, and the dashboard shows the raw and stable levels separately. This is a classification of pressure risk; predictive accuracy and lead time require a separate, labelled evaluation before numerical claims can be made.

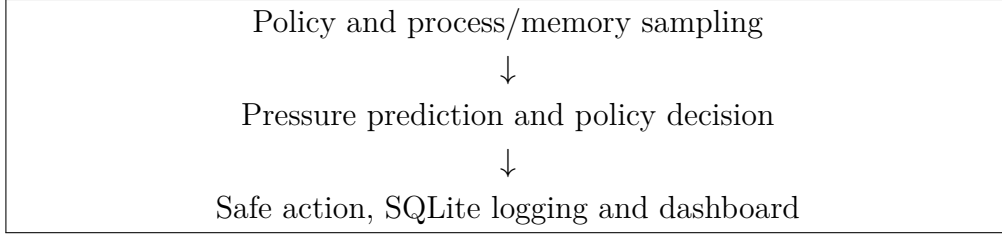
2.11 OS concept summary

OS Concept	Linux mechanism/technology	Use in this project	Purpose
Process management	Kernel process data	Samples name, status, CPU and RSS	Observe tasks
PID tracking	PID and creation time	Detects new identities; rechecks targets	Reduce reuse risk
CPU scheduling	Linux kernel scheduler	Explained as context; not implemented here	Define scope
Process priority	JSON policy labels	Selects protect or throttle decisions	Favor exam work
Memory management	RAM and cgroup metrics	Measures usage and growth	Detect strain
PSI	Memory stall metrics	Adds contention data to prediction	Detect stalls
<code>/proc</code>	Virtual kernel filesystem	Exposes process and system information	Inspect state
Signals	<code>SIGTERM</code> ; <code>SIGKILL</code> fallback	Optional guarded termination	Control process
cgroup v2	Managed memory controls	Optionally limits or protects a group	Control memory
Thread synchronization	Monitor thread and <code>RLock</code>	Protects shared dashboard state	Consistent reads
SQLite logging	SQLite WAL and exports	Persists events and samples	Audit evidence

3 System Design and Methodology

3.1 Architecture

The service follows a repeated pipeline:



`run.py` starts the Flask application. `ExamService` owns the monitor, predictor, action manager, cgroup manager, and logger. The browser polls JSON endpoints and renders process, pressure, and evaluation state. The service listens on loopback by default; this is a local teacher view, not a remotely secured deployment.

3.2 Policy and decision method

At exam start, the policy is loaded again and a new session is established. Classification precedence is protected, blocked, allowed, then unknown. The decision engine is a pure function of a process snapshot, the policy, and stable pressure. Protected or critical-priority entries are protected; blocked entries receive the configured blocked action; unknown entries are logged. Under High or Critical pressure, high-priority allowed entries receive a protect recommendation and low-priority allowed entries receive a throttle recommendation. Other allowed entries are left alone. The labels refer to the project policy rather than Linux nice values. A cooldown avoids repeatedly recording the same action for the same process identity.

3.3 Sampling and baseline

The default policy samples every two seconds; the safe demonstration policy uses one second. At the first sample, existing allowed programs become the quiet baseline, but already-running blocked and unknown processes remain candidates. Thereafter newly observed identities are evaluated. When pressure reaches High or Critical, existing processes are also reconsidered for priority-sensitive actions. Sampling means short-lived processes can be missed between intervals. Detection latency should be measured from exported events rather than assumed equal to the configured interval.

3.4 Safety boundary

The default mode is `detect_only`. Its logged success describes successful recording of a simulated decision, not successful enforcement. The optional `enforce` path has guards for dangerous PIDs and stale identities and should be exercised only in a disposable VM snapshot with a harmless named workload. The program does not provide authentication, privilege separation, or a production-safe remote control plane.

4 Implementation

4.1 Modules and responsibilities

Module	Responsibility
<code>process_monitor.py</code>	Process snapshots and new-identity detection
<code>memory_monitor.py</code>	System/cgroup memory, growth, PSI, and OOM-event sampling
<code>policy.py</code>	JSON loading, validation, classification, and policy priority
<code>predictor.py</code>	Raw/stable pressure levels, hysteresis, score, explanations
<code>decision_engine.py</code>	Side-effect-free action choice
<code>action_manager.py</code>	Detect-only records or guarded signal/cgroup actions
<code>cgroup_manager.py</code>	Managed-group readiness and memory controls
<code>event_logger.py</code>	SQLite samples, events, summaries, retention
<code>service.py</code>	Background lifecycle, synchronization, monitoring pipeline
<code>web.py</code>	Local dashboard APIs and evidence exports

4.2 Data and interfaces

`ProcessSnapshot` keeps PID, name, executable, creation time, CPU use, RSS, memory percentage, and status. `MemorySnapshot` holds totals, growth, PSI averages, source, and optional cgroup data. `PredictionResult` stores raw/stable levels, a score, confidence, and explanation. `Decision` keeps the action and rationale. The web layer exposes status, process, event, and export routes; `/export/events.csv`, `/export/samples.csv`, and a JSON report support reproduction. The dashboard uses periodic polling, not server-pushed event updates.

5 Experiments and Evaluation

5.1 Environment and reproducible procedure

The intended environment is Kali Linux in VirtualBox with Python 3, a virtual environment, and the dependencies in `requirements.txt`. A follow-up terminal screenshot shows checkout 235e343, kernel 6.19.14+kali-amd64, 3.8 GiB total memory reported by `free -h`, and 33 passed from `python -m pytest -q`. The session exports themselves do not record the commit or kernel, so this screenshot cannot independently establish the exact checkout used when that earlier session ran. Before each experiment, record the branch commit, Python and kernel versions, VM RAM allocation, policy file, sample interval, and mode. Start a fresh exam session, collect CSV/JSON exports, and preserve screenshots with timestamps. Do not compare sessions whose hardware or policy differs without noting those differences. The safe demonstration policy keeps the dashboard browser available and blocks only a harmless named workload.

Listing 1: Safe setup and local demonstration

```
git clone -b fix/evaluation-readiness https://github.com/najifa-raksanda/
  OsProject.git
cd OsProject
python3 -m venv .venv
```

```
source .venv/bin/activate
pip install -r requirements.txt
python -m pytest -q
EXAM_POLICY=config/demo_policy.json python run.py
```

Open <http://127.0.0.1:5000> in *Kali's* browser, select Start Exam, and leave the server terminal running. In a second Kali terminal, run:

Listing 2: Harmless named workloads

```
cp "$(command -v python3)" /tmp/exam-blocked
/tmp/exam-blocked tools/blocked_demo.py
# After recording evidence, stop it with Ctrl+C.
cp "$(command -v python3)" /tmp/exam-low
/tmp/exam-low tools/memory_stress.py --total-mb 300 --step-mb 10
```

The 300 MB test is bounded; it may increase growth or risk without reaching High pressure on every VM. Do not promise a throttle event unless the measured threshold is reached. Remove temporary executable copies after testing.

5.2 Experiment matrix

Scenario	Expected observation	Evidence
Baseline	Ordinary allowed tasks; stable memory samples	Sample CSV, process list
Blocked workload	exam-blocked classified blocked; detect-only terminate recommendation	Event CSV, PID, screenshot
Unknown workload	An unlisted process classified unknown and logged	Event CSV
Memory growth	Usage/growth/PSI/score observed; pressure level depends on VM	Sample CSV, plot
Priority decision	Under measured High/Critical pressure, low/high labels select different recommendations	Event CSV and policy
Optional enforcement	Harmless target, VM snapshot, verified exit or cgroup membership	Action result and Linux files

5.3 Metrics and interpretation

Let t_c be process creation time and t_e the event timestamp. Detection latency is $L_d = 1000(t_e - t_c)$ milliseconds for a newly observed process. The loop duration measures collection, decision, action, and logging time; it should be compared with the configured sampling interval. Report sample count, event count, blocked-detection count, monitor CPU/RSS, pressure distribution, peak memory/growth/PSI/risk, and attempted-action outcomes. An event count is not a count of unique applications: one application can spawn multiple processes and repeated events. Detect-only results must not be included as successful kernel-action attempts.

5.4 Results and evidence status

The exported session `c4482580ced8` used the “Safe Evaluation Demonstration” policy in `detect_only` mode on 14 September 2026 (UTC timestamps in the CSV). Its JSON session report recorded 191 samples, 210 policy events, one blocked detection, and 209 unknown-process events. These are event counts, not unique programs. All 191 samples had a stable Normal pressure classification; peak memory use was 39.8%, peak growth was 5.196 MiB/s, peak memory PSI `some avg10` was 0.0%, and peak risk was 21/100. The blocked `exam-blocked` event in the event CSV had a recorded latency of 1,302 ms and the message “Detect-only: would terminate PID 4041”. No kernel action was attempted. This confirms classification and safe logging for that workload, not actual termination or cgroup enforcement.

The source files are `report-c4482580ced8.json`, `samples-c4482580ced8.csv`, and `events-c4482580ced8.csv`. The sample and event CSV files contain 189 samples and 207 events, respectively, for the same session ID. Their smaller counts suggest they were exported slightly before the JSON summary, so the JSON totals are used in Table 2. The JSON’s average detection latency of 121,111 ms covers many already-running unknown processes at exam start and should *not* be presented as new-process response time. The blocked event’s 1,302 ms is the relevant observed example, not a statistically reliable average. Mean monitor-loop duration was 108 ms, but one loop took 5,775 ms, exceeding the one-second demonstration sampling interval. This outlier requires investigation before claiming consistently timely detection. Peak monitor RSS was 56.36 MiB; average reported monitor CPU was 6.894% during this short session. The data do not demonstrate High pressure, throttle/protect enforcement, or prediction accuracy. The follow-up Kali test screenshot shows 33 passing automated tests, but it does not verify cgroup enforcement.

Table 2: Observed session results from exported JSON and CSV

Measure	Observed value
Session ID / policy mode	<code>c4482580ced8</code> / <code>detect_only</code>
Follow-up Kali checkout / kernel	<code>235e343</code> / <code>6.19.14+kali-amd64</code>
VM total RAM / automated tests	3.8 GiB / 33 passed
Samples / events (later JSON export)	191 / 210
Blocked / unknown event counts	1 / 209
Stable pressure distribution	Normal: 191 samples
Blocked-workload detection latency	1,302 ms (one event only)
Mean / maximum monitor-loop duration	108 ms / 5,775 ms
Average monitor CPU / peak RSS	6.894% / 56.36 MiB
Peak memory use / growth	39.8% / 5.196 MiB/s
Peak memory PSI <code>some</code> / risk score	0.0% / 21 out of 100
Attempted enforcement actions	0; detect-only logging only

6 Challenges and Discussion

Process names are convenient but weak identifiers, and PID reuse creates a safety risk; creation-time checks reduce this risk. Sampling reduces implementation complexity but

may miss short-lived tasks. A baseline must not hide a blocked process already running when Exam Mode starts. Hysteresis prevents a single transient metric from repeatedly changing pressure state. Concurrent monitoring and dashboard reads require synchronization and persistent logging. Finally, cgroup readiness and actual cgroup action are distinct: the latter needs permissions, a target process, and verification. These engineering decisions favor interpretable behavior and safe classroom demonstrations over aggressive automatic intervention.

6.1 Individual contributions

The following proposed allocation uses student roll numbers and the workstreams in the earlier draft. It gives each member a distinct technical responsibility while recognizing shared integration work. *The team must confirm or correct this allocation against its actual contribution log before submitting it as a factual record.* No personal names are repeated here.

Student roll	Proposed contribution
202414010	Project integration and system architecture; coordination of evaluation, branch integration, and report preparation
202414021	Process policy, classification, safe action decisions, and enforcement-safety review
202414031	Memory sampling, PSI and predictor evaluation, and experiment analysis
202414039	Dashboard, exports, evidence preparation, and demonstration documentation
Shared work	Requirements, integration review, tests, debugging, presentation, and report review; confirm actual participation

7 Limitations

These are known constraints and opportunities for improvement, not claims that the tested prototype failed:

- Process names are weaker identifiers than verified executable paths, hashes, or digital signatures; stronger executable verification is not yet implemented.
- The local dashboard has no authentication or role-based authorization, no CSRF protection, and no strong tamper resistance. It must not be exposed publicly.
- Thresholds are manually configured and may need adjustment for VM RAM, kernel behavior, and workload mix.
- CPU and I/O PSI, swap pressure, and thermal pressure are not fully integrated into the prediction model.
- cgroup behavior depends on kernel version, controller availability, hierarchy delegation, and user permissions. Detect-only mode cannot establish enforcement effectiveness.
- Controlled experiments on a limited environment may not generalize to other hardware or Linux configurations. A larger labelled dataset is needed to quantify prediction accuracy.
- Flask’s built-in development server is suitable for local testing, not public production deployment.

8 Conclusion

Predictive Exam-Aware Resource Manager connects exam process discovery, identity tracking, resource sampling, memory-pressure detection, explainable rule-based prediction, policy-violation decisions, evidence recording, optional guarded process control, and a live local teacher dashboard. It applies Linux process interfaces, `/proc`, PSI, signals, cgroup v2, synchronization, and persistent SQLite logging. Policy priority influences application-level decisions but does not replace CPU scheduling. The default detect-only behavior keeps destructive control disabled while preserving evidence for review. Measured benefits and enforcement behavior must be stated only for sessions whose exports and Linux observations are retained.

9 Future Work

- Add authentication, teacher/admin role-based authorization, CSRF protection, policy locking, and stronger tamper-evident audit storage.
- Verify executables through canonical paths, hashes, or signatures; add teacher-approved exceptions for legitimate exam processes.
- Introduce a narrowly scoped privileged cgroup helper and test it across Linux distributions and kernel configurations.
- Calibrate adaptive machine-aware thresholds; include CPU PSI, I/O PSI, swap pressure, and thermal information where available.
- Collect more diverse labelled sessions and evaluate forecasting lead time, false alarms, and accuracy before considering statistical or machine-learning methods.
- Expand automated integration tests and CI, event-driven dashboard updates, session archival, and safe production deployment with a production-grade WSGI server.

A Reproduction Checklist

Record the exact repository commit, policy JSON, VM specifications, kernel version, Python version, dependencies, and test results. Run the safe demo in detect-only mode, export the event/sample CSV and JSON report, and take labelled screenshots. For a later session, update Table 2 only after reconciling the new exports. For optional enforcement, take a VM snapshot first; use only the named harmless workload, verify process exit or cgroup membership independently, and return to detect-only mode immediately. Do not treat a readiness label or an action recommendation as proof of kernel enforcement.

References

- [1] MIST, *0-Knowing the Course*, Operating Systems course outline, slide 8, supplied course material, 2026.
- [2] *Predictive Exam-Aware Resource Manager*, project repository, branch `fix/evaluation-readiness`, <https://github.com/najifa-raksanda/0sProject>.
- [3] Linux man-pages project, *proc(5)*, <https://man7.org/linux/man-pages/man5/proc.5.html>.

- [4] Linux man-pages project, *sched(7)* and *nice(2)*, <https://man7.org/linux/man-pages/man7/sched.7.html>.
- [5] Linux man-pages project, *signal(7)*, <https://man7.org/linux/man-pages/man7/signal.7.html>.
- [6] Linux Kernel Documentation, *Pressure Stall Information*, <https://docs.kernel.org/accounting/psi.html>.
- [7] Linux Kernel Documentation, *Control Group v2*, <https://docs.kernel.org/admin-guide/cgroup-v2.html>.
- [8] SQLite Documentation, *Write-Ahead Logging*, <https://www.sqlite.org/wal.html>.